

Introducing Context-Fabric:

Scalable Corpus Analysis for the Agentic Era

Cody Kingham

Aug 2026

Abstract

For ten years, Text-Fabric (TF) has served as a powerful tool for corpus analysis, underlying an entire generation of linguistic and exegetical research. However, TF has always been burdened by two problems: 1) it is resource intensive, consuming large amounts of RAM; and 2) it requires knowledge of Python or a query language to use it. These frictions have limited its usefulness to the philological community, which is not known for its computational prowess! Agentic AI, however, now makes it possible to solve the skill-gap. To maximize this opportunity, TF must become light and agile, capable of running on resource-constrained systems, with the ability to scale to many corpora at once. To address this, I introduce Context-Fabric (CF), a memory-efficient re-write of TF designed for the agentic era. Using memory-mapped arrays to store the corpus graph, CF achieves up to a 92% reduction in RAM consumption, with as much as $5.3\times$ more efficient scaling for multi-corpus loads. This is accomplished without sacrificing search speeds, which are 3% *faster* overall, with direct feature look up improving 26%. To take advantage of this power, CF comes with an MCP server optimized for AI agents. This makes it possible to conduct complex, high-level research without knowing a jot or tittle of Python.

1 The Legacy of Text-Fabric

In 1977, Eep Talstra founded the Werkgroep Informatica at the Vrije Universiteit Amsterdam (today the Eep Talstra Centre for Bible and Computer, or “ETCBC”) with the purpose of using computers to analyze the Hebrew Bible. Over the next fifty years, those efforts yielded a rich grammatical database of Biblical Hebrew. For most of this period, the dataset was stored privately on the Werkgroep’s internal server.¹ It was not until 2014 that Dirk Roorda, a research scientist on the [†?] project, published the dataset online as an XML dump in Linguistic Annotation Framework (LAF) [†cite]. LAF models texts as graphs. The various linguistic units like clauses and phrases can be captured as nodes with features, and their relations as edges [†cite <https://link.springer.com/article/10.1007/s10579-014-9268-1>]. Alongside the XML dump, Roorda published a Python package called LAF-Fabric, which could walk the nodes and edges to query their features [†Roorda]. By choosing Python, Roorda simultaneously introduced Biblical Hebrew to the burgeoning fields of Data Science and Machine Learning, which had begun to widely adopt Python as the default scripting language. One could, within the same program, collect statistics on a given grammatical pattern and then apply complex statistical algorithms with tools like Numpy, Pandas, Scikit Learn, and others [†cite]. These new capabilities were soon picked up by enthusiastic researchers who leveraged them for a whole host of studies. And they brought new emphasis on data-driven, reproducible results [† Synvar, Kalkan, Kingham, Naaijer, Jacobs, Erwich, Glanz and team, etc. etc.].

In 2016, Roorda improved on the idea by releasing Text-Fabric (TF) [†Roorda]. TF dramatically reduced the disk storage needed for the graph data by replacing in-line XML with a

¹It was also integrated into several paid software tools such as [†...CITE].

stand-off plain-text format, the `.tf` file. It made the graph data modular so that each `.tf` file contributed a single node feature or edge relationship. If one only wanted to analyze verb tense, for instance, he could choose to load only the `vt` feature.² The `.tf` format is a single column of data, with each row indexed by its node identifier. This strips away the syntactic overhead required by the XML format, reducing disk space and load times. Roorda envisioned that this new nimble format, along with a host of user-friendly visualization tools, would catalyze a new category of scholar, the “programming theologian”.

The vision of the computational theologian was certainly ambitious, and it dovetailed well with the ETCBC’s heritage; but its success was always hampered by one minor problem: learning to code is hard. And that difficulty was compounded by the right-brained tendencies of humanities students who were more comfortable writing exegetical summaries than loops and conditions. That is not to say the idea failed. Indeed, the study of syntax is closely aligned to left-brained tendencies, and many enterprising scholars followed the Roordian path. But the dream’s true fulfillment—of a humanities field radically transformed by a *scientific* approach—simply could not be realized given the skill gap.

Such was the state of things until 2022, when OpenAI announced on Twitter the release of a new public demo of its Large Language Model (LLM), ChatGPT [†cite]. This is the “AD” moment that will, most likely, come to separate the eras of manually-written and machine-generated code. The first beachhead was autocomplete, which could generate entire functions, fit for purpose, on the fly. Yet, most code continued to be written by hand within closely-supervised IDEs. For many engineers, the true moment of surrender came around December 2025, when Anthropic released its `Opus 4.5` model alongside its *Claude Code* tool. Here was a tool which could reliably generate entire software packages in one go, complete with tests and documentation. Since then, the trend towards agentic-authored code has sharply accelerated with the encouragement of additional models and tools (e.g. Codex, Grok Build, open source weights) and competitive pressure.

The AI revolution now makes it possible to fully realize Roorda’s vision, closing the skill gap that made Text-Fabric inaccessible for most. To maximize this opportunity, several transformations to Text-Fabric are required. First, corpus search must become *deployable* to a production environment. This means the ability to serve search from a persistent web server for thousands of requests at a time. This will enable AI agents to search remotely from any sandbox environment they happen to be running in. This requires a change to the way TF accesses the text graph. The current approach loads all nodes and edges into RAM, consuming a significant amount of memory. In production, especially under high loads, this is untenable. Memory mapping offers an alternative where graph data can be stored on disk and accessed only when needed. This method is already used by a number of performance-sensitive machine learning tools like Pytorch and Tensorflow [†cite]. Second, Text-Fabric must be enhanced to automatically build and return corpora descriptions that can be consumed by AI agents. These descriptions are crucial for agents to know what nodes, edges, and features are available for searching. Ideally, these descriptions would be completely corpus-agnostic so that they work out-of-the-box with pre-existing corpora. Additionally, Text-Fabric should return a syntax guide for running TF queries. The guide should be optimized for progressive discovery, so that agents can drill down to the specific patterns they require while ignoring the rest.

These requirements reach deep into Text-Fabric’s data model and radically transform what is possible. For this reason, and in the spirit of TF’s previous evolution from LAF-Fabric, I have chosen to rename and re-release this tool as Context-Fabric (CF). The name is a nod to the critical role of context engineering in agentic applications. The package is released under the same MIT license under a new shared organization, open for pull requests. CF implements a memory mapped data model using Numpy’s `memmap` function with `.npy` files, replacing TF’s in-memory Python dictionaries. Several optimizations are applied to ensure that the latency

²Plus some required node type and slot edge relations.

penalty of disk streaming is compensated for. In the end, CF pulls slightly ahead of TF for search speeds. Meanwhile, RAM usage drops sharply, and load times go from several seconds to nearly instantaneous. For corpus description and search syntax, CF provides a set of API methods which automatically analyze and sample the text graph, giving the agent a full map along with illustrative examples. This works even with various transcription formats. The search guide is implemented with different “menus”, allowing the agent to drill down to specific sub-topics as needed. These methods are wired to an optional MCP server (Model Context Protocol) which ships alongside CF.

2 Migrating TF’s Data Model

2.1 The Text as Graph

Text-Fabric’s fundamental data model is the “text as graph”. A corpus is conceived of as a collection of nodes (objects of interest), which may have edges (relationships). Different types of attributes, or features, can be stored on nodes and edges. Figure †TODO illustrates the organization. All nodes in the graph have an object type feature, **otype**. The only obligatory **otype** is the slot, which defines the atomic units (e.g. words, letters) which comprise the text. There is a corresponding, obligatory edge value from slots to other nodes, which is called **oslots**. This edge defines the containment relationship between a given node and its slots. For example, node 503201 in Figure †TODO *contains* slots 1 and 2, and therefore has an **oslots** edge extending to both nodes. Containment between non-slot nodes (e.g. phrase in clause) is automatically inferred from slot intersection, and need not be encoded separately.³ Descriptive attributes, or features, can be stored on both nodes and edges. For example, the first phrase node can have a **typ** feature with a value of PP, for “prepositional phrase type”. Likewise, embedded clauses may have an edge to another clause with the name of **rela** and a value of **Attr** for “attributive clause”. Text-Fabric itself is agnostic about which kinds of nodes and edges a corpus encodes, with the exception of the aforementioned obligatory types. The text-as-graph model is powerful in its flexibility and expressiveness. For this reason, it is desirable to retain it.

2.2 TF’s Python Dictionaries

In Python, dictionaries are container classes that provide interactions with a hash table. In plain terms, they efficiently route a given key to a given value, and are used for storing data as key-value pairs. Thus, given a dictionary named **oslots**, one can use a node number to very quickly look up the corresponding slot values. This architecture underlies Text-Fabric’s corpus loading algorithm, which populates a collection of dictionaries into memory. The approach has the advantage of being very fast, with the tradeoff of massive memory consumption. Large corpora can consume gigabytes. The ETCBC’s **.tf** Hebrew Bible, for example, takes up to †TODO GB of RAM when fully unfolded. The in-memory dictionary based storage is the critical component that needs to be changed to make loading and usage efficient.

2.3 Adapting to Memory Mapped Arrays

An alternative way to store key-value pairs is with linked arrays.

Memory-mapped I/O (**mmap**) maps file contents directly into a process’s virtual address space [13]. Rather than explicitly reading data into buffers, the program accesses file contents through pointers, and the OS transparently pages data in and out as needed.

NumPy’s **memmap** function provides a Python interface for memory-mapping array data [19]. Context-Fabric stores all corpus data as **.npy** files and loads them with **mmap_mode='r'** (read-only), enabling safe multi-process sharing.

³The arrows in the figure, in this sense, do not represent directly represent edge relations.

The challenge is converting Text-Fabric’s Python-native data structures to memory-mappable formats while preserving the same Python interface that existing code expects.

2.3.1 Dense Arrays for Node Features

Node features with single values per node map directly to numpy arrays. The storage format depends on the feature’s value type:

Integer features (verse numbers, word positions) store values directly in appropriately-sized dtypes (`uint8`, `uint16`, `uint32`).

Categorical features with small value sets use index encoding. The node type feature (`otype`), for example, maps each node to one of a handful of type names (“word”, “verse”, “chapter”, etc.). Since corpora typically have fewer than 256 node types, these indices fit in a `uint8` array:

```
1 # Type names: ["word", "verse", "chapter", ...]
2 # otype[node_id] = index into type_names
3 otype = np.array([0, 0, 0, ..., 1, 1, 2, ...], dtype=np.uint8)
4
5 # Lookup: 0(1)
6 node_type = type_names[otype[node_id]] # -> "word"
```

Listing 1: Categorical feature storage

This is conceptually similar to the string pool pattern (described next), but optimized for small, fixed value sets.

2.3.2 String Pools for Text Features

String-valued features (glosses, lexemes, morphological tags) present two challenges. First, Python strings cannot be memory-mapped directly. Second, not every node has a value for every feature—Text-Fabric represents these as `None`, but NumPy integer arrays have no native missing-value representation (unlike floats, which have `NaN`).⁴

Context-Fabric addresses both with a string pool pattern. Unique strings are stored once in an array, with a separate index array mapping each node to its string’s position. For missing values, we reserve a sentinel index: `0xFFFFFFFF` (the maximum `uint32` value, $2^{32} - 1$). This value cannot be a valid index—it would require over 4 billion unique strings—so encountering it unambiguously signals “no value.” This preserves the distinction between an empty string (a valid value at some index) and a truly absent value.

```
1 MISSING = 0xFFFFFFFF # Sentinel for "no value" (max uint32)
2
3 # Feature "gloss" for 5 nodes: ["king", "the", "king", None, "house"]
4 strings = np.array(["king", "the", "house"], dtype=object) # Unique values
5 indices = np.array([0, 1, 0, MISSING, 2], dtype=np.uint32) # Per-node index
6
7 def lookup(node):
8     idx = indices[node]
9     return None if idx == MISSING else strings[idx]
10
11 lookup(2) # -> "king"
12 lookup(3) # -> None (missing value)
```

Listing 2: String pool structure

The `indices` array is memory-mapped (`uint32`, one entry per node). The `strings` array is small—most features have far fewer unique values than nodes—and loads into memory. For BHSa’s `gloss` feature, 426,590 words map to roughly 9,000 unique glosses.

⁴While float arrays could use `NaN` for missing values, storing indices as floats wastes memory (float64 is 8 bytes vs. `uint16`’s 2 bytes for most features), requires type conversion on every lookup, and is semantically misleading since indices are inherently integers.

2.3.3 Compressed Sparse Row Format

Some corpus data involves variable-length sequences. Consider `oslots`, which maps each non-slot node to the word positions it contains: a 3-word phrase contains slots [5, 6, 7], while a 20-word verse contains [100, 101, ..., 119]. In Python, Text-Fabric stores this as a tuple of tuples—but Python tuples are pointers to objects scattered across memory, which cannot be memory-mapped.

Context-Fabric solves this with Compressed Sparse Row (CSR) format: concatenate all sequences into one flat `data` array, and use a separate `indptr` (index pointer) array to record where each sequence begins and ends:

```

1 # oslots for 3 nodes: phrase contains [5,6,7], clause contains [5,6,7,8], verse
  contains [5..12]
2 # Concatenate all slots into one array:
3 data = [5, 6, 7, 5, 6, 7, 8, 5, 6, 7, 8, 9, 10, 11, 12]
4
5 # Record boundaries (one entry per node, plus final endpoint):
6 indptr = [0, 3, 7, 15]
7
8 # Lookup: node i's slots are data[indptr[i] : indptr[i+1]]
9 phrase_slots = data[0:3] # -> [5, 6, 7]
10 clause_slots = data[3:7] # -> [5, 6, 7, 8]
11 verse_slots = data[7:15] # -> [5, 6, 7, 8, 9, 10, 11, 12]

```

Listing 3: CSR format for oslots

Both arrays are flat numpy arrays that can be memory-mapped. The `indptr` array has length $N + 1$ (one boundary per node, plus the final endpoint). Empty sequences cost nothing: when `indptr[i] == indptr[i+1]`, the slice is empty.

For edge features with associated values (e.g., linguistic dependency weights), `CSRArrayWithValues` adds a parallel values array:

```

1 # Edge feature "distance": node 0 -> {10: 100, 20: 200}, node 1 -> {}, node 2
  -> {30: 300}
2 # (node 0 connects to nodes 10 and 20 with distances 100 and 200)
3 indptr = [0, 2, 2, 3] # Boundaries
4 indices = [10, 20, 30] # Target nodes
5 values = [100, 200, 300] # Associated values (e.g., distances)
6
7 def get_edges(node):
8     start, end = indptr[node], indptr[node + 1]
9     return indices[start:end], values[start:end]
10
11 get_edges(0) # -> ([10, 20], [100, 200]) node 0's targets and distances
12 get_edges(1) # -> ([], []) node 1 has no edges
13 get_edges(2) # -> ([30], [300]) node 2 connects to 30

```

Listing 4: CSR with values

All arrays use `uint32` dtype and load via `mmap_mode='r'` (read-only), enabling safe multi-process sharing without copy-on-write overhead.

2.3.4 The .cfm Format

Context-Fabric's compiled format (`.cfm`) organizes data into a directory structure:

```

1 corpus/ # Corpus directory (e.g., bhsa/)
2 .cfm/ # Hidden compiled-format subdirectory
3 meta.json # Corpus metadata
4 warp/
5 otype.npy # Node types (uint8)
6 oslots_indptr.npy # CSR boundaries
7 oslots_data.npy # CSR slot data

```

```

8     computed/
9         order.npy           # Traversal order
10        rank.npy            # Node ranks
11        levup_indptr.npy    # Embedder CSR
12        levup_data.npy
13    features/
14        sp.npy              # Part of speech (dense)
15        gloss_idx.npy       # String pool indices
16        gloss_strings.npy   # Unique strings
17    edges/
18        mother_indptr.npy    # Edge CSR
19        mother_data.npy

```

Listing 5: .cfm directory structure

2.3.5 Optimizing Search Performance

The problem: paging into on-disk storage is slow. To compensate, Context-Fabric must improve on Text-Fabric’s search algorithm. As it turns out, memory-mapping with vectors also makes possible vectorized lookup, with major speed gains on certain operations. In TF search, feature-based lookup is linear ($O(n)$ time complexity):

```

1 for node in candidate_nodes:
2     if feature.value(node) == target_value:
3         result.add(node)

```

Listing 6: Per-node iteration (original)

Instead of linear search we use vectorized lookup, which is $O(1)$:

```

1 node_array = np.array(list(candidate_nodes))
2 values = feature_data[node_array - 1]
3 mask = values == target_index
4 result = set(node_array[mask])

```

Listing 7: Vectorized filtering

The vectorized strategy yields nearly an order of magnitude lower latency ($8.2\times$), translating to 26% faster feature lookups in real-world benchmarks.

However, tests continued to show significant slowdowns on searches involving containment of node types. For example, looking for a phrase which occurs within a particular clause resulted in substantially increased search latency, due to page faults while accessing the disk. Thus, the decision was made to include an optional in-memory load for pre-calculated node containment features (i.e. LevUp and LevDown [†check spelling] from Text-Fabric). For the BHSA, this costs 100 MB but yields a $1.7\times$ speedup for containment search. This offsets the page-fault penalty and brings CF to parity with TF for containment search.

2.4 Benchmarking and Results

To quantify Context-Fabric’s improvements, I conducted systematic benchmarks across memory efficiency, multi-corpus scaling, and query latency. This section describes the methodology; results follow in Section 3.

All benchmarks were conducted on:

- **Hardware:** Apple M1 Pro, 8 cores, 32 GB RAM, NVMe SSD
- **OS:** macOS Darwin 24.5.0 (arm64)
- **Python:** 3.13.11
- **NumPy:** 2.4.0

- **Text-Fabric:** 13.0.19
- **Context-Fabric:** 0.5.0

Production Linux deployments may show different absolute numbers but similar relative performance characteristics.

2.5 Test Corpora

Benchmarks span 10 corpora representing three orders of magnitude in size, from 1.6 MB to 1.1 GB. Table 1 lists the corpora in order of increasing size.

Table 1: Test corpora used in benchmarks, ordered by TF cache size.

Corpus	Description	TF Cache Size
CUC [11]	Copenhagen Ugaritic Corpus	1.6 MB
Tischendorf [9]	Tischendorf 8th Edition Greek NT	34 MB
SyrNT [7]	Syriac New Testament	52 MB
Peshitta [6]	Syriac Old Testament	55 MB
Quran [10]	Quranic Arabic Corpus	73 MB
SP [8]	Samaritan Pentateuch	147 MB
LXX [3]	Septuagint	268 MB
N1904 [4]	Nestle 1904 Greek NT	319 MB
DSS [5]	Dead Sea Scrolls	936 MB
BHSA [2]	Biblia Hebraica (ETCBC)	1.1 GB

This diversity enables analysis of how memory efficiency scales with corpus size. The BHSA (largest corpus) provides richly annotated linguistic data for the Hebrew Bible, containing 1,446,831 total nodes, 426,590 word-level nodes, 13 node types, 109 node features, and 6 edge features.

Each benchmark configuration runs 10 independent measurement runs following 2 warmup runs (excluded from statistics). Measurements use subprocess isolation via `multiprocessing.spawn` to ensure clean memory state, with explicit `gc.collect()` before measurement.⁵

Operating systems provide several metrics for quantifying process memory [15]. **Resident Set Size (RSS)** measures physical memory pages mapped to a process, including private allocations and shared pages [12]. **Unique Set Size (USS)** counts only pages unique to a process, excluding shared memory. **Proportional Set Size (PSS)** divides shared pages proportionally among processes [14].

We use RSS for all measurements. While RSS double-counts shared pages in multi-process scenarios, this “worst case” view provides transparency: the numbers represent actual memory pressure on the system. USS would dramatically undercount Context-Fabric’s memory-mapped data (potentially showing near-zero for file-backed pages), while PSS is unavailable on macOS and fluctuates as processes start and stop.

To ensure accurate measurements uncontaminated by prior allocations, each scenario runs in an isolated subprocess:

```

1 def _measure_cache_load(source: str, result_queue):
2     """Runs in a clean spawned subprocess."""
3     # Load corpus from cache
4     fabric = Fabric(locations=source)
5     api = fabric.load()

```

⁵Load time refers to the time from initiating corpus load to having a usable API handle, measured using Python’s `time.perf_counter()`.

```

6
7     # Measure total RSS after loading
8     gc.collect()
9     total_rss = psutil.Process().memory_info().rss
10    result_queue.put(total_rss)
11
12 # Main process spawns clean subprocess
13 ctx = multiprocessing.get_context('spawn')
14 p = ctx.Process(target=_measure_cache_load, args=(source, queue))
15 p.start()

```

Listing 8: Subprocess isolation pattern

This pattern ensures:

1. No residual memory from compilation or prior measurements
2. Consistent baseline across Text-Fabric and Context-Fabric
3. Reproducible results across runs

2.6 Deployment Scenarios

We evaluate three scenarios representing common deployment patterns:

2.6.1 Single Process

A single Python process loads the corpus from cache and serves requests. This represents interactive research use or single-threaded batch processing.

Measurement: Total RSS of subprocess after cache load.

2.6.2 Spawn Mode (4 Workers)

Four independent worker processes, each loading the corpus from scratch. This simulates multiprocessing with `spawn` context—the default on macOS and the only option on Windows.⁶

Measurement: Sum of RSS across all 4 worker processes.

2.6.3 Fork Mode (4 Workers)

A main process pre-loads the corpus, then forks 4 worker processes. Workers inherit the loaded corpus via copy-on-write semantics. This simulates production deployments like `gunicorn -preload`. Fork mode is the default on Linux systems in Python 3.13 and earlier.⁷

Measurement: Main process RSS + sum of worker RSS.

Note: For fork mode, summing RSS across processes may double-count shared copy-on-write pages. This is intentional—we report the “worst case” RSS sum rather than attempting to deduplicate shared pages, which would require USS and reintroduce the transparency issues discussed above.

⁶Windows lacks a `fork()` system call entirely. On macOS, `fork()` is available but Python 3.8+ defaults to `spawn` because forking processes that use certain Apple frameworks (Cocoa, Core Foundation) can cause crashes. Production deployments can explicitly use `fork` via `multiprocessing.get_context('fork')` when the application avoids these frameworks.

⁷Python 3.14 (October 2025) changed the default to `forkserver` on Linux. Applications requiring fork must explicitly request it via `multiprocessing.get_context('fork')`.

2.7 Query Performance Methodology

Query latency was measured using 100 curated search templates on the BHSA corpus, categorized into four types:

- **Lexical queries** (30): Feature-based filtering (e.g., find all verbs, filter by part of speech)
- **Structural queries** (30): Pattern matching across hierarchical structures
- **Quantified queries** (20): Queries using quantifiers (<, >, ...)
- **Complex queries** (20): Multi-constraint queries combining relations and features

Each query executes 50 times (5 runs $\times$ 10 iterations per run), with warmup iterations excluded. We report mean latency, standard deviation, and 95% confidence intervals.

2.8 Statistical Analysis

Results report mean values with 95% confidence intervals computed from 10 measurement runs per configuration. Linear regression models characterize scaling behavior, reporting slope, intercept, and R^2 values. Memory reduction percentages are computed as $(M_{TF} - M_{CF})/M_{TF} \times 100$.

3 Results

The benchmark suite covers three dimensions of performance: memory efficiency across 10 diverse corpora, multi-corpus scaling behavior, and query latency across 100 representative queries. This section presents each dimension in turn.

3.1 Memory Efficiency Across Corpora

Table 2 presents memory consumption for single-process corpus loading across all 10 benchmarked corpora, ordered by size. The results demonstrate a clear relationship between corpus size and memory reduction: smaller corpora exhibit modest reductions (20% for CUC at 1.6 MB), while larger corpora show dramatic improvements (92% for BHSA at 1.1 GB). The average memory reduction across all corpora is 65% ($\pm 21\%$).

Table 2: Memory efficiency comparison across 10 corpora in single-process mode. Corpora ordered by TF memory footprint. Load time speedup calculated as TF load time / CF load time.

Corpus	TF (MB)	CF (MB)	Reduction	Load Speedup
CUC	164	131	20%	$0.52\times$
Tischendorf	304	146	52%	$1.62\times$
Quran	401	187	54%	$1.29\times$
Peshitta	404	173	57%	$0.96\times$
Syrnt	515	145	72%	$3.08\times$
SP	644	174	73%	$4.25\times$
LXX	1,395	388	72%	$1.25\times$
N1904	1,566	411	74%	$2.41\times$
DSS	3,233	337	90%	$6.24\times$
BHSA	6,292	524	92%	$12.91\times$
Mean	1,492	262	65%	$3.45\times$

The distribution shows that memory reduction scales approximately with corpus size. For the smallest corpus (CUC), Context-Fabric achieves only 20% reduction because the baseline overhead of Python’s runtime and library imports dominates the measurement. For the largest corpus (BHSA), reduction reaches 92%—a 12-fold decrease in memory footprint. Figure 1 visualizes this relationship.

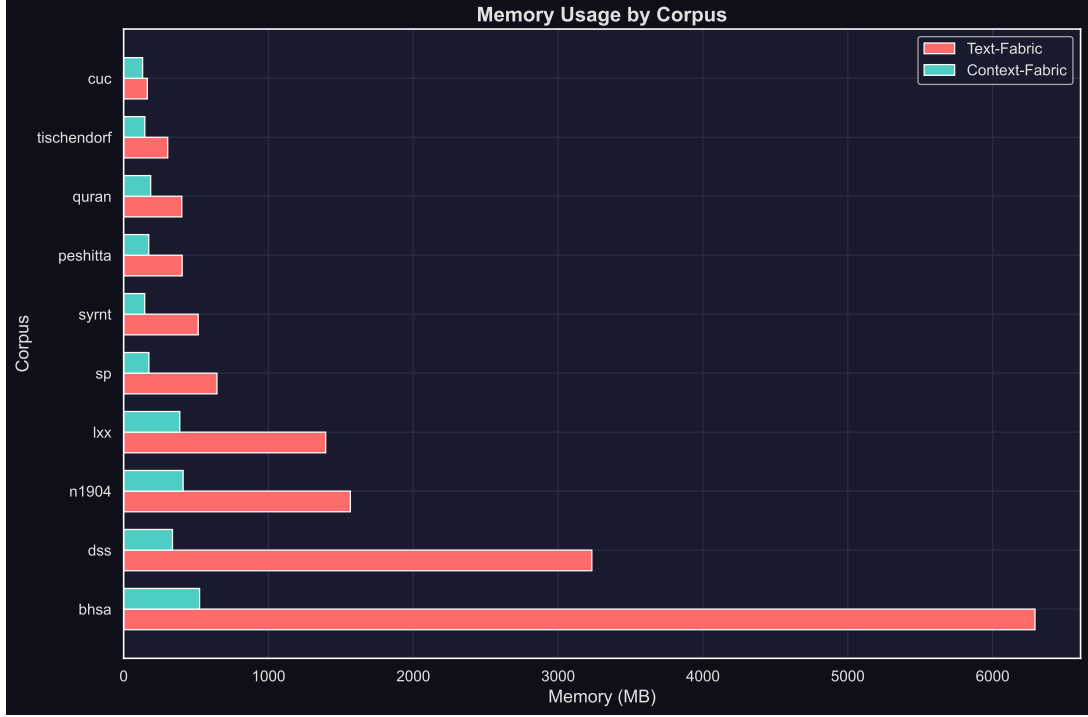


Figure 1: Memory consumption comparison across 10 corpora, sorted by Text-Fabric memory footprint. Error bars show 95% confidence intervals over 10 measurement runs.

The load time results exhibit more variance. Context-Fabric achieves substantial speedups for larger corpora ($12.9\times$ for BHSA, $6.2\times$ for DSS), but shows neutral or slightly slower load times for smaller corpora. This pattern reflects a fundamental tradeoff: Context-Fabric’s memory-mapped architecture incurs fixed initialization costs that dominate for small datasets but amortize effectively as corpus size grows.

3.2 Multi-Corpus Scaling Analysis

A key advantage of memory-mapped architecture emerges when loading multiple corpora simultaneously—a common requirement for comparative linguistic analysis. Table 3 summarizes the progressive loading results, where corpora are loaded incrementally in order of increasing size.

Note that the step 10 TF mean (5,529 MB) is anomalously lower than step 9 (6,088 MB). This probably results from Python’s garbage collector triggering non-deterministically at high memory: two of five runs showed $\sim 1,750$ MB drops mid-measurement, while others showed expected growth. The resulting variance (± 949 MB for TF vs. ± 7 MB for CF) reflects this unpredictability rather than true memory reduction.

Polynomial regression reveals distinct scaling characteristics. Both implementations fit quadratic models better than linear (R^2 improves from ~ 0.83 to ~ 0.98), but the critical difference is the quadratic coefficient: TF grows as $126n^2$ MB while CF grows as only $19n^2$ —a $6.5\times$ difference in compounding overhead. For practical corpus counts (≤ 20), CF’s weak quadratic term is negligible; TF’s is not. The linear approximation (TF: 677 MB/corpus, CF: 127 MB/corpus) captures first-order behavior but understates TF’s compounding overhead at scale.

Table 3: Memory consumption during progressive multi-corpus loading. Values represent mean total RSS (MB) across 5 runs after loading the specified number of corpora.

Corpora Loaded	TF (MB)	CF (MB)	Reduction	Ratio
1 (CUC)	163	130	20%	1.3×
2 (+Tischendorf)	306	146	52%	2.1×
3 (+Syrnt)	641	161	75%	4.0×
4 (+Peshitta)	866	204	76%	4.2×
5 (+Quran)	1,070	259	76%	4.1×
6 (+SP)	1,531	300	80%	5.1×
7 (+LXX)	2,717	551	80%	4.9×
8 (+N1904)	4,044	791	80%	5.1×
9 (+DSS)	6,088	968	84%	6.3×
10 (+BHSA)	5,529	1,348	76%	4.1×

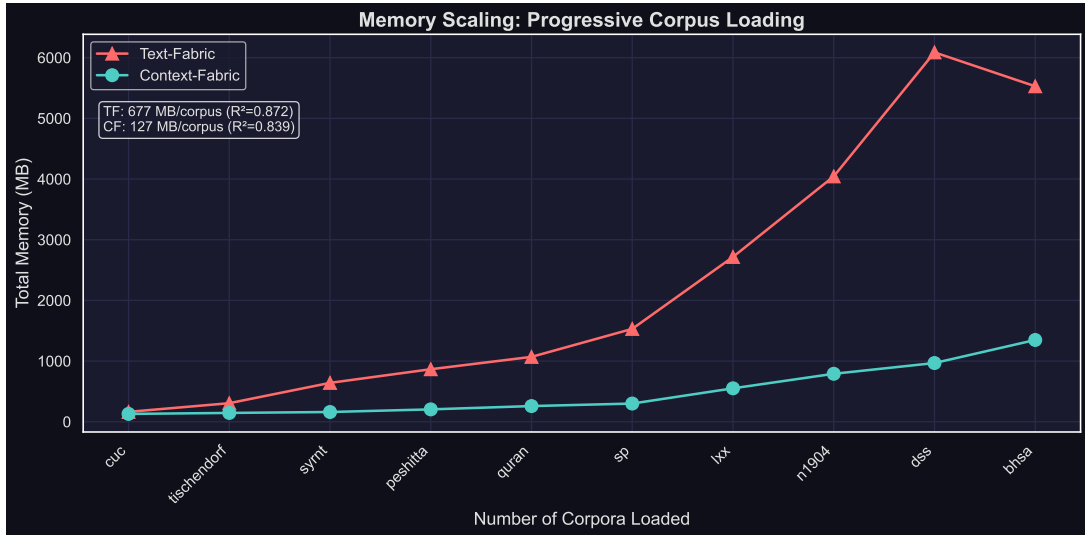


Figure 2: Memory scaling during progressive corpus loading. Quadratic fits show TF grows as $126n^2$ MB (significant compounding overhead) vs. CF at $19n^2$ (6.5× smaller quadratic coefficient).

The extrapolated predictions are telling. At 10 corpora, the model predicts TF memory of 5,344 MB versus CF memory of 1,057 MB. At 50 corpora (a realistic scenario for comprehensive cross-linguistic research), the gap widens dramatically: 32,443 MB for TF versus 6,137 MB for CF. What appears as a 4–5 $\times$ advantage at small scale compounds to an architectural necessity at large scale.

A major limitation of Text-Fabric’s architecture is its reliance on Python’s native data structures. While these provide flexibility for interactive research, they impose substantial memory overhead in multi-corpus deployments. Each dictionary resize, each reference count update, each garbage collection tracking entry contributes to compounding overhead. The effect is superlinear scaling—each additional corpus adds not only its own data but also contributes to fragmentation and memory management overhead across the entire process.

3.3 Query Latency Performance

Query performance was evaluated across 100 queries spanning four categories: lexical lookups (30 queries), structural pattern matching (30 queries), quantified searches (20 queries), and complex multi-constraint queries (20 queries). Table 4 summarizes performance by category.⁸

Table 4: Mean query latency (ms) by category. Positive speedup indicates CF is faster; negative indicates CF is slower.

Category	Queries	TF (ms)	CF (ms)	Speedup
Lexical	30	221.5	164.0	+26%
Structural	30	179.2	190.5	−6%
Quantified	20	297.3	286.1	+4%
Complex	20	534.5	554.9	−4%
Overall	100	286.0	278.2	+3%

The results reveal a nuanced performance profile. Context-Fabric demonstrates a 26% speedup on lexical queries, where memory-mapped access to indexed data structures provides direct benefits. However, CF shows 4–6% slower performance on structural and complex queries that require traversing hierarchical relationships or evaluating multiple constraints.

Figure 4 shows the cumulative distribution of query latencies across all 100 queries. At the median (p50), both implementations perform similarly. At the 95th percentile, TF shows higher latency on lexical queries while CF shows higher latency on complex structural queries. This tradeoff reflects fundamental architectural differences: Text-Fabric’s in-memory Python objects enable fast pointer traversal for structural queries, while Context-Fabric’s memory-mapped arrays require additional indirection for hierarchical navigation.

Overall, Context-Fabric achieves a 3% speedup across all queries while maintaining the dramatic memory advantages documented above. For workloads dominated by lexical queries (common in corpus linguistics), the speedup reaches 26%. For workloads requiring extensive structural traversal, users may observe modest latency increases that are typically acceptable given the memory savings.

3.4 Summary of Results

Table 5 consolidates the key findings.

The results demonstrate that Context-Fabric achieves its primary design goal: enabling multi-corpus research within constrained memory environments without sacrificing query performance. The 65% average memory reduction, combined with 5.3 $\times$ better scaling characteristics,

⁸All query benchmarks were run with CF’s embedding preloading enabled (the default). See Section ?? for discussion of the preloading trade-off.

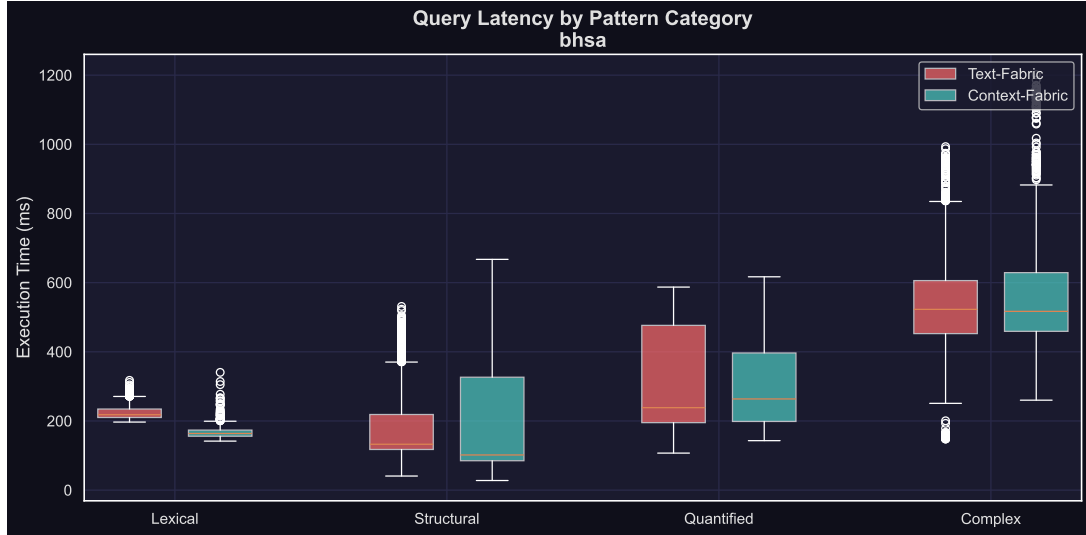


Figure 3: Query latency distribution by category. Boxes show interquartile range; whiskers extend to $1.5 \times$ IQR.

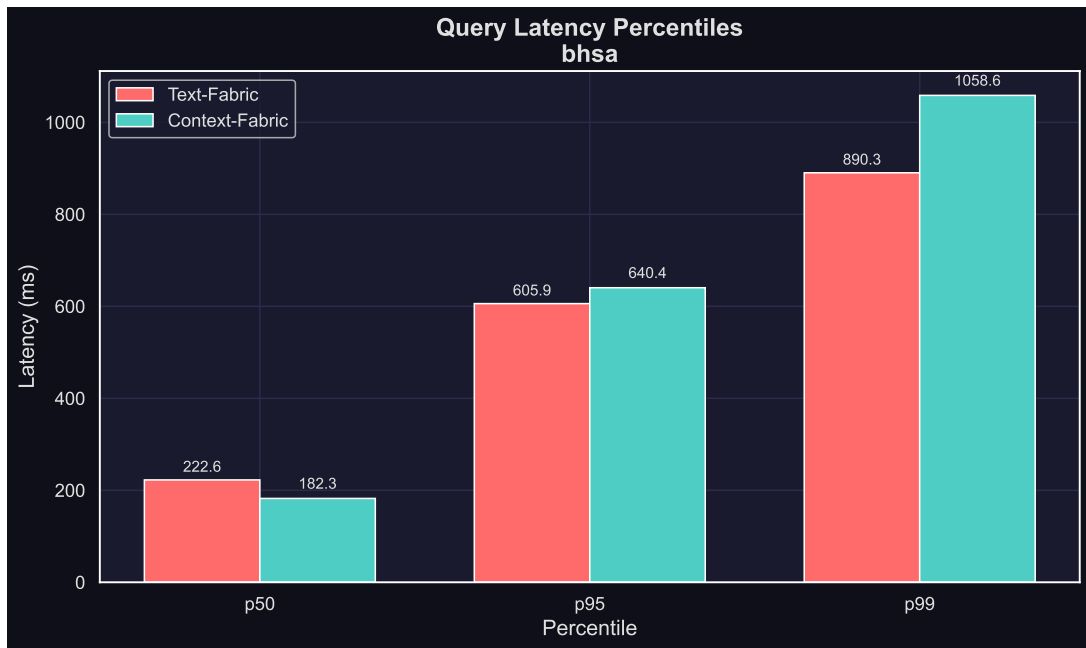


Figure 4: Query latency percentiles (p50, p95, p99) across all 100 benchmark queries.

Table 5: Summary of benchmark results comparing Text-Fabric and Context-Fabric.

Metric	Result	Interpretation
Single-corpus memory	65% reduction (mean)	20–92% range by corpus size
Multi-corpus scaling	127 vs 677 MB/corpus	5.3× scaling advantage
10-corpus deployment	1,348 vs 5,529 MB	76% total reduction
Load time (large corpora)	Up to 12.9× faster	BHSA loads in 0.55s vs 7.1s
Lexical query latency	26% faster	Memory-mapped index access
Structural query latency	6% slower	Indirection overhead
Overall query latency	3% faster	Balanced workload

makes previously impractical research configurations feasible. A researcher with 16 GB of RAM can now work with corpora that would have required 40+ GB under the traditional architecture.

4 Populating Agentic Context

4.1 Automatic Corpus Description

4.2 Search Guide

5 Using Context-Fabric

5.1 Installation

Context-Fabric is available on PyPI:

```
1 pip install context-fabric
```

Source code and documentation are available at <https://github.com/Context-Fabric/context-fabric>.

5.2 Reproducibility

All benchmarks can be reproduced using the `cfabric-benchmarks` package:

```
1 # Install and download corpora
2 pip install cfabric-benchmarks
3 python -m cfabric_benchmarks.corpora.download
4
5 # Run full benchmark suite
6 cfabric-bench full --corpora-dir .corpora
```

Results are saved to `benchmark_results/` with timestamps. Configuration, environment metadata, and raw measurements are preserved for reproducibility.

5.3 Usecases

6 Future Direction

Context-Fabric is being re-written in Rust, which will bring an even further step-change in performance.

References

- [1] Roorda, D. (2018). Text-Fabric: Text representation for linguistic annotation. *Research Data Journal for the Humanities and Social Sciences*, 3(1), 1–23.
- [2] van Peursen, W. Th., Sikkel, C., & Roorda, D. (2015). Hebrew Text Database BHSA. DANS. <https://doi.org/10.17026/dans-z6y-skyh>. Data curated by the Eep Talstra Centre for Bible and Computer, Vrije Universiteit Amsterdam.
- [3] Center for Biblical Languages and Computing. (2024). Septuagint (Rahlfs’ LXX Edition 1935) in Text-Fabric. Based on Eliran Wong’s RLXX1935 dataset. GitHub: <https://github.com/CenterBLC/LXX>
- [4] Center for Biblical Languages and Computing. (2024). Nestle 1904 Greek New Testament in Text-Fabric. GitHub: <https://github.com/CenterBLC/N1904>

- [5] Jacobs, J., Naaijer, M., & Roorda, D. (2017). Dead Sea Scrolls in Text-Fabric. Data provided by Martin Abegg. Developed as part of the CACCHT project. GitHub: <https://github.com/ETCBC/dss>
- [6] van Peursen, W. Th., Veldman, G. J., Sikkels, C., Vlaardingerbroek, H., & Roorda, D. (2018). ETCBC/peshitta: Syriac Old Testament (Peshitta) in Text-Fabric. Zenodo. <https://doi.org/10.5281/zenodo.1464757>
- [7] Vlaardingerbroek, H. & Roorda, D. (2017). Syriac New Testament in Text-Fabric. Source data from SEDRA database by George A. Kiraz and James W. Bennett. GitHub: <https://github.com/ETCBC/syrnt>
- [8] Naaijer, M., Højgaard, C. C., Schorch, S., & Ehrensverd, M. (2020). Samaritan Pentateuch in Text-Fabric. Developed as part of the CACCHT project. GitHub: <https://github.com/DT-UCPH/sp>
- [9] Kingham, C. (2018). Tischendorf 8th Edition Greek New Testament in Text-Fabric. Based on Ulrik Sandborg-Petersen’s corpus. GitHub: https://github.com/codykingham/tischendorf_tf
- [10] van Lit, C. & Roorda, D. (2017). Quranic Arabic Corpus in Text-Fabric. Source data from the Quranic Arabic Corpus and Tanzil project. GitHub: <https://github.com/q-ran/quran>
- [11] Højgaard, C. C., Naaijer, M., Ehrensverd, M., et al. (2020). Copenhagen Ugaritic Corpus in Text-Fabric. Developed as part of the CACCHT project. GitHub: <https://github.com/DT-UCPH/cuc>
- [12] Wikipedia contributors. (2024). Resident set size. *Wikipedia, The Free Encyclopedia*. Retrieved from https://en.wikipedia.org/wiki/Resident_set_size
- [13] Wikipedia contributors. (2024). Memory-mapped file. *Wikipedia, The Free Encyclopedia*. Retrieved from https://en.wikipedia.org/wiki/Memory-mapped_file
- [14] Baeldung. (2024). Understanding memory usage in Linux. *Baeldung on Linux*. Retrieved from <https://www.baeldung.com/linux/resident-set-vs-virtual-memory-size>
- [15] Beyer, D., Löwe, S., & Wendler, P. (2019). Reliable benchmarking: requirements and solutions. *International Journal on Software Tools for Technology Transfer*, 21(1), 1–29.
- [16] Langner, T., & Beyer, D. (2021). Mess: Memory consumption benchmarking made easy. *arXiv preprint arXiv:2106.08235*. Retrieved from <https://arxiv.org/abs/2106.08235>
- [17] SQLite Consortium. (2024). Memory-Mapped I/O. *SQLite Documentation*. Retrieved from <https://sqlite.org/mmap.html>
- [18] Crotty, A., Leis, V., & Pavlo, A. (2022). Are You Sure You Want to Use MMAP in Your Database Management System? *Proceedings of the 12th Annual Conference on Innovative Data Systems Research (CIDR)*. Retrieved from <https://db.cs.cmu.edu/mmap-cidr2022/>
- [19] NumPy Developers. (2024). numpy.memmap. *NumPy Documentation*. Retrieved from <https://numpy.org/doc/stable/reference/generated/numpy.memmap.html>
- [20] Rossant, C. (2018). Processing large NumPy arrays with memory mapping. *IPython Cookbook, 2nd Edition*. Retrieved from <https://ipython-books.github.io/48-processing-large-numpy-arrays-with-memory-mapping/>

- [21] van Gompel, M. & van den Bosch, A. (2016). Efficient n-gram, skipgram and flexgram modelling with Colibri Core. *Journal of Open Research Software*, 4(1), e30. <https://doi.org/10.5334/jors.105>
- [22] Hardie, A. (2012). CQPweb—combining power, flexibility and usability in a corpus analysis tool. *International Journal of Corpus Linguistics*, 17(3), 380–409.
- [23] Kilgarriff, A., Baisa, V., Bušta, J., Jakubíček, M., Kovář, V., Michelfeit, J., Rychlý, P., & Suchomel, V. (2014). The Sketch Engine: ten years on. *Lexicography*, 1(1), 7–36.